

SQLite CRUD Operations in Dart

Database Programming Tutorial

Learning Objectives

By the end of this session, you will understand:

-  **CRUD** operations (Create, Read, Update, Delete)
-  **SQLite** database integration in Dart
-  **Model-Service** architecture pattern
-  **Unit testing** for database operations
-  **Best practices** for database programming

What is CRUD?

CRUD represents the four basic operations in database management:

Operation	SQL Command	Purpose
Create	INSERT	Add new records
Read	SELECT	Retrieve existing records
Update	UPDATE	Modify existing records
Delete	DELETE	Remove records

These operations form the foundation of most database applications!

Project Structure Overview

```
sqlite/
├── lib/
│   ├── models/
│   │   └── foobar.dart                # Data model
│   └── services/
│       └── foobar_crud_sqlite.dart    # CRUD operations
├── test/
│   └── foobar_crud_test.dart          # Unit tests
├── data/
│   └── foobar.db                      # SQLite database (auto-generated)
└── doc/
    └── crud_tutorial.md               # This presentation
```

Separation of Concerns: Models define data structure, Services handle database operations.

The FooBar Model

```
class FooBar {  
  String foo;    // Text field  
  int bar;       // Number field  
  
  FooBar({required this.foo, required this.bar});  
  
  // Convert database row to object  
  factory FooBar.fromRow(Row row) {  
    return FooBar(  
      foo: row['foo'] as String,  
      bar: row['bar'] as int,  
    );  
  }  
  
  // Convert object to database format  
  Map<String, dynamic> toMap() => {  
    'foo': foo,  
    'bar': bar,  
  };  
}
```

Database Table Structure

Our SQLite table stores FooBar objects:

```
CREATE TABLE IF NOT EXISTS foobars (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  -- Auto-generated ID  
  foo TEXT NOT NULL,                     -- String field  
  bar INTEGER NOT NULL,                   -- Integer field  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Key Points:

- `PRIMARY KEY AUTOINCREMENT` creates unique IDs automatically
- `NOT NULL` ensures required fields have values
- `CURRENT_TIMESTAMP` automatically sets creation time

CREATE Operation

Purpose: Add new records to the database

```
Future<int> create(FooBar foobar) async {  
    final db = await database;  
  
    final stmt = db.prepare('''  
        INSERT INTO foobars (foo, bar)  
        VALUES (?, ?)  
    ''');  
  
    final result = stmt.execute([foobar.foo, foobar.bar]);  
    stmt.dispose(); // Always clean up prepared statements!  
  
    return db.lastInsertRowId; // Returns the new record's ID  
}
```

Usage Example:

```
final newFooBar = FooBar(foo: 'Hello', bar: 42);
```

READ Operations

Read All Records

```
Future<List<FooBar>> readAll() async {  
    final db = await database;  
    final ResultSet resultSet = db.select('SELECT * FROM foobars ORDER BY id');  
    return resultSet.map((row) => FooBar.fromRow(row)).toList();  
}
```

Read by ID

```
Future<FooBar?> readById(int id) async {  
    final db = await database;  
    final stmt = db.prepare('SELECT * FROM foobars WHERE id = ?');  
    final result = stmt.select([id]);  
    stmt.dispose();  
  
    return result.isEmpty ? null : FooBar.fromRow(result.first);  
}
```


READ Operations - Search Example

Search by Text Field

```
Future<List<FooBar>> findByFoo(String foo) async {  
    final db = await database;  
  
    final stmt = db.prepare('SELECT * FROM foobars WHERE foo LIKE ?');  
    final result = stmt.select(['%$foo%']); // % = wildcard  
    stmt.dispose();  
  
    return result.map((row) => FooBar.fromRow(row)).toList();  
}
```

Usage:

```
// Find all records containing "test"  
final results = await crudService.findByFoo('test');  
// Finds: "test", "testing", "latest", etc.
```

UPDATE Operation

Purpose: Modify existing records

```
Future<bool> update(int id, FooBar foobar) async {  
    final db = await database;  
  
    final stmt = db.prepare('''  
        UPDATE foobars  
        SET foo = ?, bar = ?  
        WHERE id = ?  
    ''');  
  
    final result = stmt.execute([foobar.foo, foobar.bar, id]);  
    stmt.dispose();  
  
    return result.changes > 0; // True if record was updated  
}
```

Usage Example:

DELETE Operations

Delete by ID

```
Future<bool> delete(int id) async {  
    final db = await database;  
  
    final stmt = db.prepare('DELETE FROM foobars WHERE id = ?');  
    final result = stmt.execute([id]);  
    stmt.dispose();  
  
    return result.changes > 0; // True if record was deleted  
}
```

Delete All (Use with caution!)

```
Future<int> deleteAll() async {  
    final db = await database;  
    final result = db.execute('DELETE FROM foobars');  
    return result.changes; // Returns number of deleted records  
}
```

Utility Operations

Count Records

```
Future<int> count() async {  
    final db = await database;  
    final result = db.select('SELECT COUNT(*) as count FROM foobars');  
    return result.first['count'] as int;  
}
```

Check if Record Exists

```
Future<bool> exists(int id) async {  
    final db = await database;  
    final stmt = db.prepare('SELECT 1 FROM foobars WHERE id = ? LIMIT 1');  
    final result = stmt.select([id]);  
    stmt.dispose();  
    return result.isNotEmpty;  
}
```

Database Connection Management

```
class FooBarCrudSQLite {
  Database? _database;
  final String _databaseName = 'foobar.db';
  final String _dataDirectory = 'data';

  Future<Database> get database async {
    if (_database != null) return _database!; // Reuse existing

    // Ensure data directory exists
    final dataDir = Directory(_dataDirectory);
    if (!await dataDir.exists()) {
      await dataDir.create(recursive: true);
    }

    String dbPath = join(_dataDirectory, _databaseName);
    _database = sqlite3.open(dbPath);
    _createTableIfNotExists(); // Ensure table exists
    return _database!;
  }

  Future<void> close() async {
    _database?.dispose(); // Always close when done!
    _database = null;
  }
}
```

Testing CRUD Operations

Test Structure Pattern

```
void main() {  
    late FooBarCrudSQLite crudService;  
  
    setUp(() async {  
        crudService = FooBarCrudSQLite();  
        await crudService.deleteAll(); // Clean slate for each test  
    });  
  
    tearDown(() async {  
        await crudService.close(); // Clean up after each test  
    });  
  
    test('CREATE: Should insert a new record', () async {  
        // Arrange: Prepare test data  
        final testFooBar = FooBar(foo: 'Hello', bar: 42);  
  
        // Act: Perform the operation  
        final id = await crudService.create(testFooBar);  
  
        // Assert: Check the results  
        expect(id, greaterThan(0));  
    });  
}
```

Testing Examples

Test CREATE Operation

```
test('CREATE: Should insert a new FooBar record', () async {  
  final testFooBar = FooBar(foo: 'Hello', bar: 42);  
  final id = await crudService.create(testFooBar);  
  
  expect(id, greaterThan(0));  
  
  // Verify by reading back  
  final retrieved = await crudService.readById(id);  
  expect(retrieved!.foo, equals('Hello'));  
  expect(retrieved.bar, equals(42));  
});
```

Test UPDATE Operation

```
test('UPDATE: Should update existing record', () async {  
  final id = await crudService.create(FooBar(foo: 'Original', bar: 1));  
  final updated = await crudService.update(id, FooBar(foo: 'Updated', bar: 2));  
  expect(updated, equals(FooBar(foo: 'Updated', bar: 2)));  
});
```

Best Practices

1. Always Use Prepared Statements

```
// ✓ GOOD: Prevents SQL injection
final stmt = db.prepare('SELECT * FROM foobars WHERE id = ?');
final result = stmt.select([id]);

// ✗ BAD: Vulnerable to SQL injection
final result = db.select('SELECT * FROM foobars WHERE id = $id');
```

2. Handle Null Results

```
// ✓ GOOD: Safe null handling
final foobar = await crudService.readById(id);
if (foobar != null) {
  print('Found: ${foobar.foo}');
} else {
  print('Not found');
}
```


Best Practices (Continued)

3. Always Dispose Resources

```
// ✓ GOOD: Clean up prepared statements
final stmt = db.prepare('SELECT * FROM foobars');
final result = stmt.select();
stmt.dispose(); // Important!

// ✓ GOOD: Close database when done
await crudService.close();
```

4. Use Transactions for Multiple Operations

```
Future<void> bulkInsert(List<FooBar> foobars) async {
  final db = await database;

  db.execute('BEGIN TRANSACTION');
  try {
    for (final foobar in foobars) {
```

Common Pitfalls to Avoid

1. SQL Injection Vulnerabilities

```
// ❌ DANGEROUS: Never do this!  
db.select("SELECT * FROM foobars WHERE foo = '$userInput'");  
  
// ✅ SAFE: Use prepared statements  
stmt.select([userInput]);
```

2. Resource Leaks

```
// ❌ BAD: Forgot to dispose  
final stmt = db.prepare('SELECT * FROM foobars');  
stmt.select();  
// Missing: stmt.dispose();  
  
// ❌ BAD: Forgot to close database  
final crud = FooBarCrudSQLite();  
// Use crud...
```

Error Handling Strategies

Graceful Error Handling

```
Future<FooBar?> safeFindById(int id) async {  
  try {  
    return await readById(id);  
  } catch (e) {  
    print('Error reading record $id: $e');  
    return null; // Return safe default  
  }  
}
```

```
Future<bool> safeUpdate(int id, FooBar foobar) async {  
  try {  
    return await update(id, foobar);  
  } catch (e) {  
    print('Error updating record $id: $e');  
    return false; // Indicate failure  
  }  
}
```

Running the Code

1. Run the Application

```
cd /path/to/sqlite/project  
dart run lib/main.dart
```

2. Run the Tests

```
dart test
```

3. Run Specific Test

```
dart test test/foobar_crud_test.dart
```

4. Run with Verbose Output

```
dart test --reporter=expanded
```

Real-World Applications

Student Management System

```
class Student {  
    String name;  
    int age;  
    String email;  
  
    // Similar structure to FooBar but for real data  
}
```

Inventory Management

```
class Product {  
    String name;  
    int quantity;  
    double price;  
  
    // CRUD operations for managing inventory
```

Advanced Topics (Future Learning)

Database Relationships

- **One-to-Many:** User → Multiple Orders
- **Many-to-Many:** Students ↔ Courses
- **Foreign Keys:** Maintaining data integrity

Performance Optimization

- **Indexing:** Speed up queries
- **Query Optimization:** Efficient SQL
- **Connection Pooling:** Manage resources

Data Migration

- **Schema Versioning:** Handle database changes

Summary & Key Takeaways

What You've Learned

- CRUD operations are fundamental to database programming
- SQLite provides reliable local database storage
- Prepared statements prevent SQL injection
- Testing ensures code reliability
- Resource management is crucial

Next Steps

1. Practice with different data models
2. Implement error handling strategies
3. Add database indexes for performance
4. Explore database relationships

Assignment Ideas

Beginner Level

1. **Library System:** Books with title, author, ISBN
2. **Contact Manager:** Names, phone numbers, emails
3. **Todo List:** Tasks with priorities and due dates

Intermediate Level

1. **Expense Tracker:** Categories, amounts, dates
2. **Grade Manager:** Students, subjects, scores
3. **Recipe Database:** Ingredients, instructions, ratings

Advanced Level

1. **E-commerce Backend:** Products, orders, customers

Resources & References

Documentation

- [SQLite Documentation](#)
- [Dart sqlite3 Package](#)
- [Dart Testing Guide](#)

Best Practices

- [Database Design Principles](#)
- [SQL Injection Prevention](#)
- [Unit Testing Best Practices](#)

Tools

- **DB Browser for SQLite:** Visual database tool

Questions & Discussion

Discussion Topics

1. When would you choose SQLite over other databases?
2. How do you handle database schema changes in production?
3. What are the trade-offs between different database approaches?

Practical Questions

1. How would you implement pagination for large datasets?
2. What happens if two operations try to modify the same record?
3. How would you backup and restore database data?

Thank you for your attention!

Ready for hands-on practice? 